Phytium飞腾

# 飞腾嵌入式 Linux
# Yocto 4.0 用户手册

（V2.7）

飞腾信息技术有限公司
www.phytium.com.cn
2026 年 03 月 02 日

**版权所有 © 飞腾信息技术有限公司 2024。保留一切权利。**

未经本公司同意，任何单位、公司或个人不得擅自复制、翻译、摘抄本文档内容的部分或全部，不得以任何方式或途径进行传播和宣传。

**商标声明**

Phytium 和其他飞腾商标均为飞腾信息技术有限公司的商标。

本文档提及的其他所有商标或注册商标，由各自的所有人拥有。

**特别提示**

本文档仅作为使用指导，飞腾对本文档内容不做任何明示或暗示的声明或保证。本文档中的所有陈述、信息和建议不构成任何明示或暗示的担保。

由于产品版本升级或其他原因，本文档内容会不定期进行更新，如有变更，恕不另行通知。

**最新技术资源**

飞腾嵌入式软件开源社区 https://gitee.com/phytium_embedded

或者访问飞腾软件开发者平台 https://service.phytium.com.cn/developer/

# 目录

1 概述 ..... 1
2 主机开发环境配置 ..... 1
2.1 硬件配置 ..... 1
2.2 一键设置开发环境（推荐） ..... 1
2.3 自行定制开发环境（不推荐） ..... 2
2.3.1 安装开发工具 ..... 2
2.3.2 获取源码并建立开发环境 ..... 2
2.4 线下代码获取 ..... 2
2.5 外部编译工具链使用 ..... 3
3 系统镜像构建 ..... 4
3.1 设置开发环境变量 ..... 4
3.2 执行构建 ..... 4
3.2.1 最小系统，phydisk 和 Xfce 桌面系统 ..... 4
3.2.2 Weston 系统 ..... 5
3.2.3 无网络环境 ..... 5
3.2.4 更换内核版本 ..... 5
3.2.5 实时内核 ..... 6
3.2.6 Xenomai 系统 ..... 6
3.2.7 EtherCAT 系统 ..... 7
3.2.8 Jailhouse 系统 ..... 8
3.2.9 支持 E2000 VPU，X100，D3000M DC，VPU，GPU ..... 8
4 phydisk 镜像的使用 ..... 8
4.1 Uboot 启动 ..... 8
4.2 UEFI 启动系统 ..... 9
5 rootfs 镜像的使用 ..... 9
5.1 Uboot ..... 9
5.1.1 镜像烧录 ..... 9

5.1.2 镜像启动 ..... 10
5.1.3 镜像启动登陆 ..... 11
5.2 UEFI ..... 11
5.2.1 镜像烧录 ..... 11
5.2.2 镜像启动登陆 ..... 15
6 制作和使用系统安装盘 ..... 15
6.1 编译 iso 镜像 ..... 15
6.2 制作安装盘 ..... 15
6.3 使用安装盘 ..... 16
6.4 系统启动 ..... 17
7 系统镜像定制 ..... 18
7.1 内核定制 ..... 18
7.2 文件系统定制 ..... 18
7.3 BSP 定制 ..... 20
7.4 软件包管理 ..... 21
8 常用软件 ..... 21
8.1 图形和显示 ..... 21
8.2 多媒体 ..... 21
9 附录 ..... 22
9.1 开发主机交叉编译环境配置 ..... 22
9.1.1 基于 Yocto 生成交叉编译工具链 ..... 22
9.1.2 下载交叉编译工具链 ..... 23
9.2 常见问题 ..... 23
9.3 参考 ..... 25
A 更新记录 ..... 26

# 1 概述

嵌入式 Linux 是在嵌入式计算机系统上运行的 Linux 系统，通常会根据系统的软硬件要求做出裁剪，以达到较低的功耗和较高的性能。

为了方便在飞腾开发板上定制嵌入式 Linux 系统，加快评估开发周期，这里提供了一种构建系统的方式—— Yocto. (https://www.yoctoproject.org/)

Yocto Project 是一个开源的协作软件，提供模板、工具和方法帮你创建定制的 Linux 系统和嵌入式产品，而无需关心硬件体系。适合嵌入式 Linux 开发人员使用。

本文档主要介绍通过 Yocto 4.0 工程如何构建针对飞腾全系列 CPU 的嵌入式 Linux 内核镜像及文件系统。用户可以使用 Yocto 4.0 项目的组件进行开发，构建，调试和测试完整的嵌入式 Linux 系统。

# 2 主机开发环境配置

## 2.1 硬件配置

a) X86 主机(非 ARM 架构 )
b) 最少 4GB 内存
c) Ubuntu22.04、Debian11 系统之一
d) 磁盘剩余空间至少 50GB

## 2.2 一键设置开发环境（推荐）

使用内置的 phyt_yocto_setenv.sh 脚本可以一步完成开发环境配置，实现了自动安装主机环境中所需的软件包、部署外部编译工具链，安装 repo 工具和设定开发目录。

```
$ git clone https://gitee.com/phytium_embedded/phytium-linux-yocto.git  -b <branch>
$ cd phytium-linux-yocto
$ source  phyt_yocto_setenv.sh
```

master 分支：支持 4.19 版本内核与 5.10 版本内核。

kernel-6.6 分支：仅支持 6.6 版本内核。

## 2.3 自行定制开发环境（不推荐）

除了使用脚本以外，还可以一步步自行定制开发环境，包括主机软件安装，repo 安装，repo 同步 Yocto 库。在这种方式下默认使用内部编译工具链，如需使用外部编译工具链，请参见“外部编译工具链使用”小节。

### 2.3.1 安装开发工具

主机系统中需要安装如下软件包：

```
$ sudo apt-get  build-essential chrpath cpio debianutils diffstat file gawk gcc git iputils-ping libacl1 liblz4-tool locales python3 python3-git python3-jinja2 python3-pexpect python3-pip python3-subunit socat texinfo unzip wget xz-utils zstd
```

### 2.3.2 获取源码并建立开发环境

飞腾嵌入式 Linux 源码可以通过 repo 或脚本与远程 gitee 代码仓库同步获取。

安装 repo 工具，

```
$ mkdir ~/bin
$ curl https://mirrors.tuna.tsinghua.edu.cn/git/git-repo > ~/bin/repo
$ export REPO_URL='https://mirrors.tuna.tsinghua.edu.cn/git/git-repo'
$ chmod a+x ~/bin/repo
$ export PATH=${PATH}:~/bin
```

创建工作目录，设置 git 和 repo 同步 Yocto 仓库，

```
$ export PATH=${PATH}:~/bin
$ mkdir <phytium-path>
$ cd <phytium-path>
$ git config --global user.name "Your Name"
$ git config --global user.email email@example.com
$ sudo ln -s /usr/bin/python3 /usr/bin/python
$ repo init -u https://gitee.com/phytium_embedded/phytium-linux-yocto.git -b master
$ repo sync --force-sync
$ source setup-env -m phytium
```

## 2.4 线下代码获取

以上两种方式都是通过网络来直接同步代码，如果没有网络条件，飞腾还提供两种 ISO 代码包，请根据自身需要选择使用，ISO 代码包请咨询 linan1284@phytium.com.cn 获取：

表 22- 1 ISO 文件

| | |
|---|---|
| phytium-linux-yocto-<version>-source-<yyyymmdd>-yocto.iso | 源码包，包含可编译的源码，客户可从源码来构建整个系统 |
| phytium-linux-yocto-<version>-cache-<yyyymmdd>-yocto.iso | Cache 安装包，包含预编译的二进制文件，可加快源码构建时的编译速度 |

源码包 ISO 包安装命令如下：

```
$ sudo mount -o loop phytium-linux-yocto-<version>-source-<yyyymmdd>-yocto.iso /mnt
$ cd /mnt
$ ./install
```

cache ISO 包安装命令如下：

```
$ sudo mount -o loop phytium-linux-yocto-<version>-cache-<yyyymmdd>-yocto.iso /mnt
$ cd /mnt
$ ./install
```

## 2.5 外部编译工具链使用

Yocto 内部编译工具链需要在编译时重新编译，会加长编译时间。可以直接使用外部编译工具链来省去编译时间。

外部边编译工具链下载地址:

https://gitee.com/phytium_embedded/phytium-rogue-umlibs/raw/develop/pre-toolchain/phytium-glibc-x86_64-meta-toolchain-aarch64-toolchain-4.0.6.sh

下载工具链到 64 位 X86 开发主机上，然后安装

```
$ sh phytium-glibc-x86_64-meta-toolchain-aarch64-toolchain-4.0.6.sh
```

并修改 local.conf 添加下列内容:

```
EXTERNAL_TOOLCHAIN = "/path/to/your/external-toolchain-installed"
TCMODE = "external-oe-sdk"
PREFERRED_PROVIDER_virtual/crypt = "libxcrypt"
PREFERRED_PROVIDER_linux-libc-headers = "glibc-external"
PREFERRED_PROVIDER_linux-libc-headers-dev = "glibc-external"
PREFERRED_PROVIDER_virtual/linux-libc-headers = "glibc-external"
```

# 3 系统镜像构建

通过源码可构建的 rootfs 镜像为：

表 3- 2 可选镜像

| 名字 | 说明 | 提供层 |
|---|---|---|
| core-image-minimal | 最小文件系统 | poky |
| core-image-weston | Wayland + Weston | poky |
| core-image-xfce | Xfce 轻量级桌面环境 | meta-phytium |
| core-image-rt | 实时文件系统 | Poky |

下面介绍如何构建系统镜像。

（注意： 请不要用 root 用户执行下列操作）

## 3.1 设置开发环境变量

```
$ cd <phytium-path> or phytium-linux-yocto-<version>-<yyyymmdd>-yocto #repo 和 ISO 方式的 path 不同
$ source setup-env -m phytium
```

## 3.2 执行构建

通过 bitbake 命令可以构建系统镜像，以下使用<rootfs_image>表示可选 rootfs 镜像的名字。

编译完成后用户可以在 tmp/deploy/images/<machine>/下找到编译好的文件，文件说明如下表。

| 镜像类型 | 说明 |
|---|---|
| Image | 内核 |
| dtb | 设备树 |
| rootfs | 文件系统 |
| phydisk.img | 包含了根文件系统、内核、设备树和 GRUB |

### 3.2.1 最小系统，phydisk 和 Xfce 桌面系统

编译系统镜像：

```
$ bitbake <rootfs_image>
```

<rootfs_image>表示 core-image-minimal,phydisk,core-image-xfce 编译 sd 卡启动的镜像 phydisk.img(目前仅支持飞腾派),需要在 conf/local.conf 中设置下列变量

```
MACHINE_FEATURES:append = " sd"
```

### 3.2.2 Weston 系统

编译 weston 的文件系统, 需要在 conf/local.conf 中设置下列变量:

```
DISTRO_FEATURES:remove = " x11"
```

编译系统镜像:

```
$ bitbake core-image-weston
```

### 3.2.3 无网络环境

在不能连接 Internet 的环境下需要使用安装的 ISO, 同时需要设置 conf / local.conf 文件中如下变量:

```
BB_NO_NETWORK = "1"
```

编译系统镜像:

```
$ bitbake <rootfs_image>
```

### 3.2.4 更换内核版本

飞腾 Linux kernel（https://gitee.com/phytium_embedded/phytium-linux-kernel.git）中有六个分支：linux-6.6，linux-6.6-rt ,linux-5.10，linux-5.10-rt，linux-4.19，linux-4.19-rt。不同版本内核的编译需匹配 phytium-linux-yocto 项目的对应配置，同时实时 RT 内核及系统镜像的构建也有特定要求，具体说明如下：

Linux 6.6 内核编译：需使用 phytium-linux-yocto 项目的 kernel-6.6 分支，通过该分支预设的编译配置，可编译 phytium Linux kernel 的 linux-6.6 及 linux-6.6-rt 分支源码。

Linux 5.10 内核编译：需使用 phytium-linux-yocto 项目的 master 分支，采用分支内默认配置即可完成编译，无需额外修改参数，可编译 phytium Linux kernel 的 linux-5.10 及 linux-5.10-rt 分支。

Linux 4.19 内核编译：需使用 phytium-linux-yocto 项目的 master 分支，在 conf/local.conf 配置文件中添加以下版本指定变量，以适配对应内核分支：

```
PREFERRED_VERSION_linux-phytium = "4.19"
```

编译内核:

```
$ bitbake virtual/kernel
```

### 3.2.5 实时内核

如果编译带 Linux 实时 RT 内核的系统镜像，需要在 conf/local.conf 文件中添加下列变量：

1) 如果编译 Linux 6.6 RT 或 Linux 5.10 RT 内核，添加：

```
PREFERRED_PROVIDER_virtual/kernel = "linux-phytium-rt"
```

2) 如果编译 Linux 4.19 RT 内核，添加：

```
PREFERRED_PROVIDER_virtual/kernel = "linux-phytium-rt"
PREFERRED_VERSION_linux-phytium-rt = "4.19"
```

编译内核：

```
$ bitbake virtual/kernel
```

编译系统镜像：

```
$ bitbake <rootfs_image>
```

### 3.2.6 Xenomai 系统

1) 编译 Xenomai cobalt 模式

使用 xenomai 内核（https://gitee.com/phytium_embedded/linux-kernel-xenomai.git），该内核有 Linux 6.6，Linux 5.10 和 Linux 4.19 三个版本, 不同版本的编译配置如下：

a) 编译 Linux 6.6 版本 xenomai 内核，需 phytium-linux-yocto 项目的 kernel-6.6 分支，在 conf/local.conf 文件中添加以下内容：

```
PREFERRED_PROVIDER_virtual/kernel = "linux-xenomai-phytium"
```

b) 编译 Linux 5.10 版本 xenomai 内核，需 phytium-linux-yocto 项目的 master 分支，在 conf/local.conf 文件中添加以下内容：

```
PREFERRED_PROVIDER_virtual/kernel = "linux-xenomai-phytium"
```

c) 编译 Linux 4.19 版本 xenomai 内核，需 phytium-linux-yocto 项目的 master 分支，在 conf/local.conf 文件中添加以下内容：

```
PREFERRED_PROVIDER_virtual/kernel = "linux-xenomai-phytium"
PREFERRED_VERSION_linux-xenomai-phytium = "4.19"
PREFERRED_VERSION_xenomai = "3.1.5"
```

执行以下命令进行编译：

```
$ bitbake core-image-rt
```

2) 编译 Xenomai mercury Preempt-RT 模式

使用实时内核（https://gitee.com/phytium_embedded/phytium-linux-kernel.git），该仓库中有 linux-6.6-rt,linux-5.10-rt 和 linux-4.19-rt 分支。

a) 编译 Linux 6.6 RT 内核，需 phytium-linux-yocto 项目的 kernel-6.6 分支，

修改 conf/local.conf 文件，添加以下内容：

```
PREFERRED_PROVIDER_virtual/kernel = "linux-phytium-rt"
```

修改 sources/meta-phytium/meta-xenomai/recipes-xenomai/xenomai/xenomai_3.3.bb，将

```
EXTRA_OECONF += " --enable-pshared --enable-smp \
        --with-core=cobalt"
```

修改为

```
EXTRA_OECONF += " --enable-pshared --enable-smp \
        --with-core=mercury"
```

b) 编译 Linux 5.10 RT 内核,需 phytium-linux-yocto 项目的 master 分支

修改 conf/local.conf 文件，添加以下内容：

```
PREFERRED_PROVIDER_virtual/kernel = "linux-phytium-rt"
```

修改 sources/meta-phytium/meta-xenomai/recipes-xenomai/xenomai/xenomai_3.2.6.bb，将

```
EXTRA_OECONF += " --enable-pshared --enable-smp \
        --with-core=cobalt"
```

修改为

```
EXTRA_OECONF += " --enable-pshared --enable-smp \
        --with-core=mercury"
```

c) 编译 Linux 4.19 RT 内核,需使用 phytium-linux-yocto 项目的 master 分支

修改 conf/local.conf 文件，添加以下内容：

```
PREFERRED_PROVIDER_virtual/kernel = "linux-phytium-rt"
PREFERRED_VERSION_linux-phytium-rt = "4.19"
PREFERRED_VERSION_xenomai = "3.1.5"
```

修改 sources/meta-phytium/meta-xenomai/recipes-xenomai/xenomai/xenomai_3.1.5.bb，将

```
EXTRA_OECONF += " --enable-pshared --enable-smp \
        --with-core=cobalt"
```

修改为

```
EXTRA_OECONF += " --enable-pshared --enable-smp \
        --with-core=mercury"
```

执行以下命令进行编译:

```
$ bitbake core-image-rt
```

### 3.2.7 EtherCAT 系统

编译 core-image-rt 文件系统镜像，默认包括 EtherCAT(https://gitee.com/phytium_embedded/ether-cat.git) 软件包. Xenomai 系统也可以支持 ethercat ,构建时设置如下参数:

```
DISTRO_FEATURES += " xenomai"
PREFERRED_PROVIDER_virtual/kernel = "linux-xenomai-phytium"
```

文件系统里,程序服务已关闭，需要手动启动服务程序：

```
$ systemctl restart ethercat.service
```

### 3.2.8 Jailhouse 系统

编译 Jailhouse(https://gitee.com/phytium_embedded/phytium-jailhouse.git)的文件系统, 需要在 conf/local.conf 中设置下列变量：

```
DISTRO_FEATURES:append = " jailhouse"
```

编译系统镜像：

```
$ bitbake core-image-xfce
```

### 3.2.9 支持 E2000 VPU，X100，D3000M DC，VPU，GPU

core-image-xfce 文件系统可以支持 E2000 VPU（视频解码器），X100, D3000M 的 DC，VPU, GPU，但代码不开源，镜像默认不支持这些功能。软件安装包获取请咨询 linan1284@phytium.com.cn

# 4 phydisk 镜像的使用

Phydisk.img 镜像包含了根文件系统、内核、设备树和 GRUB。使用 img 镜像安装系统，不需要将存储设备手动分区再拷贝文件，只要将 phydisk.img 文件写入存储设备即可。

```
$ sudo  umount /dev/sdb*
$ sudo dd if=phydisk.img of=/dev/sdb bs=1M
$ sync
```

## 4.1 Uboot 启动

安装系统后，将存储设备接到开发板，启动开发板电源，串口输出 U-Boot 命令行，设置 U-Boot 环境变量来启动系统。

SATA 盘：

```
=>setenv bootargs console=ttyAMA1,115200  audit=0 earlycon=pl011,0x2800d000
root=PARTUUID="0a52c129-7e0f-43ad-989f-d96b07ccdbb2" rw;
=>fatload scsi 0:1 0x90100000 Image;
=>fatload scsi 0:1 0x90000000 e2000q-demo-board.dtb;
=>booti 0x90100000 - 0x90000000
=>saveenv
```

U 盘：

```
=>setenv bootargs console=ttyAMA1,115200  audit=0 earlycon=pl011,0x2800d000
root=PARTUUID="0a52c129-7e0f-43ad-989f-d96b07ccdbb2" rootdelay=5 rw;
=>usb start
=>fatload usb 0:1 0x90100000 Image;
=>fatload usb 0:1 0x90000000 e2000q-demo-board.dtb;
=>booti 0x90100000 - 0x90000000
=>saveenv
```

## 4.2 UEFI 启动系统

如果是使用 phydisk.img 镜像安装系统，将存储设备接到开发板，启动开发板电源，进入 UEFI 模式,选择启动的介质,即可自动启动系统。

# 5 rootfs 镜像的使用

可以通过 Uboot 或者 UEFI 启动系统镜像。编译生成的输出包括,Image, dtb , rootfs.

**前提条件**

a) 一台基于 Linux 发行版的主机。

b) 一块硬盘（固态硬盘或者机械硬盘均可）或 U 盘均可（将制作成 UEFI 引导盘，会格式化硬盘，注意保存原有数据）。

## 5.1 Uboot

### 5.1.1 镜像烧录

以 core-image-minimal 文件系统为例。

**前提条件：**

a) 主机串口线连接板卡的串口

b) 主机安装串口调试工具（Kermit、putty、minicom 等）

**在主机端将系统镜像拷贝到 USB、SATA 或 NVME 设备：**

以主机识别设备名为/dev/sdb 为例，请按实际识别设备名更改，nvme 设备请改为 nvme 的设备名，或参考下一节（UEFI 启动）中分区和格式化步骤。

将设备分区：

```
$ sudo fdisk /dev/sdb
输入下列参数，每个参数都跟回车
p        [打印分区表]
d        [删除分区]
n        [添加新分区]
p        [主分区]
1        [第一个分区]
2048     [分区起始扇区]
+1G      [第一个分区大小]
p        [检查分区]
n        [添加新分区]
p        [主分区]
2        [第二个分区]
2200000  [分区起始扇区]
+100G    [第二个分区大小]
p        [打印分区表]
w        [将分区表写入磁盘并退出]
```

格式化设备，并将系统镜像拷贝到设备：

```
$ sudo mkfs.ext4 /dev/sdb1
$ sudo mkfs.ext4 /dev/sdb2
$ sudo mount /dev/sdb1 /mnt
$ cd <image-path>（<image-path>为 tmp/deploy/images/<machine>）
$ sudo cp Image e2000d-demo-board.dtb /mnt（以 e2000d demo 板为例，其它开发板修改为对应的设备树即可）
$ sudo umount /dev/sdb1
$ sudo mount /dev/sdb2 /mnt
$ sudo cp core-image-minimal.tar.gz /mnt
$ cd /mnt
$ sudo tar zxvf core-image-minimal.tar.gz
$ cd ~
$ sudo umount /dev/sdb2
```

### 5.1.2 镜像启动

**在开发板上设置 Uboot 环境变量：**

文件系统在 U 盘上（以 U 盘启动后识别为/dev/sda 为例，请按实际识别设备名更改 root= 参数）：

```
=>setenv bootargs console=ttyAMA1,115200  audit=0 earlycon=pl011,0x2800d000 root=/dev/sda2 rw
=>usb start
```

```
=>ext4load usb 0:1 0x90100000 Image
=>ext4load usb 0:1 0x90000000 e2000d-demo-board.dtb
=>booti 0x90100000 - 0x90000000
```

文件系统在 SATA 硬盘上（以 SATA 盘启动后识别为/dev/sda 为例，请按实际识别设备名更改 root= 参数）：

```
=>setenv bootargs console=ttyAMA1,115200  audit=0 earlycon=pl011,0x2800d000
root=/dev/sda2 rw
=>ext4load scsi 0:1 0x90100000 Image
=>ext4load scsi 0:1 0x90000000 e2000d-demo-board.dtb
=>booti 0x90100000 - 0x90000000
```

文件系统在 M.2 接口的 NVME 盘上：

```
=>setenv bootargs console=ttyAMA1,115200  audit=0 earlycon=pl011,0x2800d000
root=/dev/nvme0n1p2 rw
=>ext4load nvme 0:1 0x90100000 Image
=>ext4load nvme 0:1 0x90000000 e2000d-demo-board.dtb
=>booti 0x90100000 - 0x90000000
```

### 5.1.3 镜像启动登陆

文件系统启动后,串口控制台登陆系统,输入 root 用户名,按回车即可,没有密码.图形登陆系统,没有密码登陆界面.

## 5.2 UEFI

### 5.2.1 镜像烧录

**制作步骤**

1) 在终端下找到对应硬盘盘符，在本文档中，该硬盘为/dev/sdb，注意根据实际情况替换相应设备。
2) 在 root 用户下使用 fdisk 分区，如下所示。

```
root@sxf-OptiPlex-7070:/home/sxf# fdisk /dev/sdb
```

图 5.1 分区命令

3) 使用 p 命令查看当前分区，若存在分区，使用 d 命令删除。

```
Command (m for help): d
Partition number (1,2, default 2): 1

Partition 1 has been deleted.

Command (m for help): d
Selected partition 2
Partition 2 has been deleted.

Command (m for help):
```

图 5.2 删除分区

4) 确认删除所有分区后，输入 g 命令标记硬盘分区格式为 GPT（UEFI 分区格式）。

```
Command (m for help): g
Created a new GPT disklabel (GUID: 76710B1B-695C-CF49-96EA-5FADFDD8953A).
The old dos signature will be removed by a write command.

Command (m for help):
```

图 5.3 分区格式

5) 之后使用 n 命令开始分区，第一个分区大小为 500M，为 EFI 分区，输入命令如下，首先是分区号，默认为 1，回车即可；之后是起始扇区，回车即可；之后是尾部扇区，输入+500M，表明分区大小为 500M，扇区地址自动计算。之后使用 t 命令设置分区类型，输入 1 表明是 EFI 分区，具体操作如下图所示。

```
Command (m for help): n
Partition number (1-128, default 1):
First sector (2048-1953525134, default 2048):
Last sector, +sectors or +size{K,M,G,T,P} (2048-1953525134, default 1953525134): +500M

Created a new partition 1 of type 'Linux filesystem' and of size 500 MiB.

Command (m for help): t
Selected partition 1
Partition type (type L to list all types): 1
Changed type of partition 'Linux filesystem' to 'EFI System'.

Command (m for help):
```

图 5.4 第一个分区

6) 完成第一个分区后，在使用 n 命令创建第二个分区，操作类似，分区大小自行设定，操作如下所示。

```
Command (m for help): n
Partition number (2-128, default 2):
First sector (1026048-1953525134, default 1026048):
Last sector, +sectors or +size{K,M,G,T,P} (1026048-1953525134, default 1953525134): +20G

Created a new partition 2 of type 'Linux filesystem' and of size 20 GiB.

Command (m for help):
```

图 5.5 第二个分区

7) 其他分区操作类似，分区完成后使用 p 命令查看当前分区情况，如下所示。

```
Command (m for help): p
Disk /dev/sdb: 931.5 GiB, 1000204886016 bytes, 1953525168 sectors
Units: sectors of 1 * 512 = 512 bytes
Sector size (logical/physical): 512 bytes / 4096 bytes
I/O size (minimum/optimal): 4096 bytes / 4096 bytes
Disklabel type: gpt
Disk identifier: 76710B1B-695C-CF49-96EA-5FADFDD8953A

Device         Start        End    Sectors  Size Type
/dev/sdb1       2048    1026047    1024000  500M EFI System
/dev/sdb2    1026048   42969087   41943040   20G Linux filesystem
/dev/sdb3   42969088  462399487  419430400  200G Linux filesystem
/dev/sdb4  462399488  881829887  419430400  200G Linux filesystem
/dev/sdb5  881829888 1953525134 1071695247  511G Linux filesystem

Command (m for help): w
The partition table has been altered.
Calling ioctl() to re-read partition table.
Syncing disks.

root@sxf-OptiPlex-7070:/home/sxf#
```

图 5.6 分区结果

8) 确认没有问题后，使用 w 命令保存设置并退出，若需要删除分区，可以使用 d 命令，按照提示操作即可。

9) 分区完成后，使用 mkfs.vfat 格式化 EFI 分区，使用 mkfs.ext4 格式化其他分区，如下图所示。

```
root@sxf-OptiPlex-7070:/home/sxf# mkfs.vfat /dev/sdb1
mkfs.fat 4.1 (2017-01-24)
root@sxf-OptiPlex-7070:/home/sxf# mkfs.ext4 /dev/sdb2
mke2fs 1.44.1 (24-Mar-2018)
Creating filesystem with 5242880 4k blocks and 1310720 inodes
Filesystem UUID: 72e94b6b-7015-4783-ad07-675086b266ae
Superblock backups stored on blocks:
        32768, 98304, 163840, 229376, 294912, 819200, 884736, 1605632, 2654208,
        4096000

Allocating group tables: done
Writing inode tables: done
Creating journal (32768 blocks): done
Writing superblocks and filesystem accounting information: done

root@sxf-OptiPlex-7070:/home/sxf#
```

图 5.7 格式化分区

10) 挂载根文件系统分区，将根文件系统包拷贝到对应分区即可，飞腾提供示例根文件系统包，用户也可选用自己编译的镜像，示例如下：

```
$ sudo mount /dev/sdb1 /mnt
$ sudo mount /dev/sdb2 /mnt2
$ cd <image-path>（<image-path>为 tmp/deploy/images/<machine>）
$ sudo tar zxvf core-image-minimal.tar.gz -C /mnt2
$ sudo cp -rL /mnt2/boot/* /mnt
$ sudo cp Image /mnt
```

修改引导文件/mnt/EFI/BOOT/grub.cfg，示例文件如下所示：

```
serial --unit=0 --speed=115200 --word=8 --parity=no --stop=1
search --no-floppy --set=root -l 'boot'
default=boot
timeout=10

menuentry 'boot'{
linux /Image LABEL=boot root=/dev/sda2
}
```

主要将 root 参数修改为对应的根文件系统分区，如在本例中第二个分区为根文件系统，则将 root 参数修改为 root=/dev/sda2（默认只有一个硬盘，如果有多个硬盘，建议使用对应分区的 UUID）。

可根据需要仿照上述格式添加多个表项。

卸载分区：

```
$ sync
$ sudo umount /mnt /mnt2
```

### 5.2.2 镜像启动登陆

引导盘制作好后，在 UEFI 下将该硬盘设为默认启动盘即可运行系统。

文件系统启动后,串口控制台登陆系统,输入 root 用户名,按回车即可,没有密码;图形登陆系统,没有密码登陆界面.

# 6 制作和使用系统安装盘

飞腾提供定制的基于 X11 的桌面系统，通过本章可把飞腾定制的桌面系统编译成 ISO，制作成安装盘并安装到硬盘中，供用户快速测试使用。

## 6.1 编译 iso 镜像

1) 设置开发板环境变量

```
$ cd <phytium-path> or phytium-linux-yocto-<version>-<yyyymmdd>-yocto #repo 和 ISO 方式的 path 不同
$ source setup-env -m phytium
```

2) 将以下变量添加到 conf/local.conf 中

```
DISTRO_FEATURES:append = " anaconda-support pam"
VIRTUAL-RUNTIME_init_manager = "systemd"
DISTRO_FEATURES:append = " systemd"
DISTRO_FEATURES_BACKFILL_CONSIDERED:append = " sysvinit"
```

3) 编译系统

```
$ bitbake core-image-xfce
```

4) Xfce 系统编译完成后，在 conf/local.conf 中把上面变量去掉，把下面这些变量添加进去：

```
DISTRO = "anaconda"
INSTALLER_TARGET_BUILD = "<phytium-path>/build_phytium/tmp/deploy/images/phytium/core-image-xfce-phytium.ext4"
```

5) 编译 ISO

```
$ bitbake core-image-anaconda
```

编译完成后，ISO 位于 build_phytium/tmp-glibc/deploy/images/phytium/core-image-anaconda-phytium.iso

## 6.2 制作安装盘

使用上节编译的 ISO 文件制作安装盘，具体使用方法如下：

在主机端插入 U 盘，运行下列命令制作安装盘（请按照插入 U 盘识别的设备名来更改，下列命令以 /dev/sdb 为例）：

1) 如果使用 Uboot 启动：

```
$ sudo mount -o loop core-image-anaconda-phytium.iso media/
$ sudo mkfs.ext4 -L boot /dev/sdb1
$ sudo mount /dev/sdb1 /mnt
$ sudo rsync -a -P media/ /mnt/
$ sudo umount /mnt
```

2) 如果使用 UEFI 启动：

```
$ sudo dd if=core-image-anaconda-phytium.iso of=/dev/sdb bs=1M
```

## 6.3 使用安装盘

将安装盘插入开发板的 USB 接口，启动。

1) Uboot 启动模式下，在串口控制台设置 Uboot 环境变量：

```
=>setenv bootargs root=/dev/ram0 rootdelay=5 rw initrd=0x93000000,140M
=>usb start
=>ext4load usb 0:1 0x90100000 Image
=>ext4load usb 0:1 0x90000000 dtb/e2000d-demo-board.dtb（以 E2000d demo 板为例，其它开发板修改为对应的设备树即可）
=>ext4load usb 0:1 0x93000000 initrd
=>booti 0x90100000 - 0x90000000
```

2) UEFI 启动模式下，开机按 F2，选择 U 盘作为启动设备，然后在 GRUB 界面选择“boot graphics console”，等待进入图形安装界面。

在图型安装界面，需要做以下设置：

首先需要选择安装过程使用的语言，保持默认即可，点击 Continue，来到“INSTALLATION SUMMARY”安装主界面，这里有三个地方需要设置。

a) 选择系统的安装位置：点击 Installation Destination，选择想要将系统安装到哪个设备（USB 安装盘之外的磁盘，需要提前备份磁盘上的数据），选择设备后，点击屏幕左上角的 Done，在弹出的界面中依次点击 Reclaim space ->Delete all -> Reclaim space，表示要删除该磁盘上的所有分区和数据，然后回到了安装主界面。

b) 时区：点击 Time & Date，选择 Region: Asia, City: Shanghai，点击 Done，回到安装主界面。

c) 用户设置：点击 Root Password，输入 root 用户的密码，点击 Done；点击 User Creation 创建新用户，输入用户名密码，点击 Done，回到安装主界面。

配置完成后，点击系统安装主界面右下角的 Begin Installation，开始安装。

安装过程如果提示写 bootloader 错误，请选 Yes 选项。安装成功后，点击界面右下角的 Reboot System，重启开发板。

## 6.4 系统启动

1) Uboot 启动模式下，串口控制台可以设置通过 SATA 盘、NVME 盘或 eMMC 卡来启动系统。

如果前面选择将系统安装到 SATA 盘，则启动参数为：

```
=>setenv bootargs root=/dev/mapper/openembedded_phytium-root rootdelay=5 rw
initrd=0x93000000,140M console=ttyAMA1,115200
=>ext4load scsi 0:1 0x90100000 Image;
=>ext4load scsi 0:1 0x90000000 e2000d-demo-board.dtb;
=>ext4load scsi 0:1 0x93000000 initrd;
=>booti 0x90100000 - 0x90000000
=>saveenv
```

如果前面选择将系统安装到 NVME 盘，则启动参数为：

```
=>setenv bootargs root=/dev/mapper/openembedded_phytium-root rootdelay=5 rw
```

```
initrd=0x93000000,140M console=ttyAMA1,115200
=>ext4load nvme 0:1 0x90100000 Image;
=>ext4load nvme 0:1 0x90000000 e2000d-demo-board.dtb;
=>ext4load nvme 0:1 0x93000000 initrd;
=>booti 0x90100000 - 0x90000000
=>saveenv
```

如果前面选择将系统安装到 eMMC 卡，则启动参数为：

```
=>setenv bootargs root=/dev/mapper/openembedded_phytium-root rootdelay=5 rw
initrd=0x93000000,140M console=ttyAMA1,115200
=>ext4load mmc 0:1 0x90100000 Image;
=>ext4load mmc 0:1 0x90000000 e2000d-demo-board.dtb;
=>ext4load mmc 0:1 0x93000000 initrd;
=>booti 0x90100000 - 0x90000000
=>saveenv
```

2) UEFI 启动模式下，重新启动即可进入刚才安装的系统。

# 7 系统镜像定制

## 7.1 内核定制

```
$ bitbake -c menuconfig virtual/kernel（进入 kernel 选项配置）
```

根据需要可添加或删减内核组件。

## 7.2 文件系统定制

1) 在例子文件系统中添加软件包 -- 通过修改 build_<board>/conf/local.conf 配置文件添加软件包到镜像文件中：

```
IMAGE_INSTALL:append = " python"
PREFERRED_VERSION_python =" 3.4.0"（可指定软件包版本）
```

2) 定制用户自己的文件系统 -- 在 sources/meta-phytium/meta-bsp/recipes-core/文件夹下执行下列命令（以 custom 名字为例）：

```
$ mkdir -p meta-phytium/meta-bsp/recipes-core/custom
$ cd meta-phytium/meta-bsp/recipes-core/custom
$ vi custom.bb
```

添加下面内容到 custom.bb 文件中：

```
IMAGE_INSTALL = "packagegroup-core-x11-base package1 package2"
inherit core-image
```

添加完成后，进入相应目录进行编译：

```
$ bitbake custom
```

3) 编写新的应用软件包（以下以 hello 名字为例，recipe 涉及概念较多，下面只是简单介绍，请具体参考官方文档来编写）

a) 在 meta-phytium/meta-bsp/recipes-core/ 目录下创建 hello 文件夹。

b) 在 hello 文件夹下创建 hello.bb 文件及 hello 文件夹。

c) 在 hello 文件夹中放入 hello.c 及 Makefile 文件，hello 文件夹下目录文件如下：

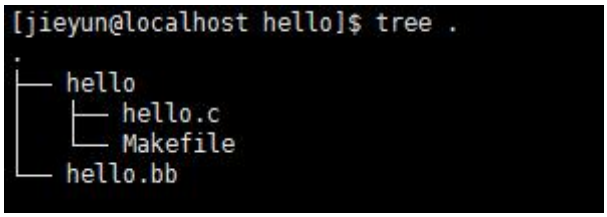


图 7.8 hello 文件夹目录结构

d) hello.bb 文件内容如下（也可以参考其他已有的 bb 文件）：

```
SUMMARY = "Simple hello application"
SECTION = "libs"
LICENSE = "MIT"
LIC_FILES_CHKSUM = "file://hello.c;md5= 0835ade698e0bcf8506ecda2f7b4f302 "
SRC_URI = "\
  file://hello.c \
  file://Makefile \
"
S = "${WORKDIR}"
do_compile() {
  make
}
do_install() {
  install -d ${D}${bindir}
  install -m 0755 ${S}/hello ${D}${bindir}
}
```

上述 md5 值可在 ubuntu 中运行 md5sum hello.c 来生成。

e) 编译 hello：

```
$ bitbake hello
```

f) 将 hello 添加到镜像中：

以添加到 core-imge-minimal 中为例，查找 core-image-minimal 的 bb 文件位置：

```
$ find ./ -name "*core-image-minimal*"
```

```
[jieyun@localhost rpi-yocto]$ find ./ -name "*core-image-minimal*"
./poky/meta/recipes-core/images/core-image-minimal-dev.bb
./poky/meta/recipes-core/images/core-image-minimal-initramfs.bb
./poky/meta/recipes-core/images/core-image-minimal-mtdutils.bb
./poky/meta/recipes-core/images/core-image-minimal.bb.bak
./poky/meta/recipes-core/images/core-image-minimal.bb
```

图 7.9 查找 core-image-minimal.bb

编辑 core-image-minimal.bb 文件，在 IMAGE_INSTALL 后面添加程序 hello

```
SUMMARY = "A small image just capable of allowing a device to boot."

IMAGE_INSTALL += "packagegroup-core-boot ${CORE_IMAGE_EXTRA_INSTALL} \
                 hello
                 "

IMAGE_LINGUAS = " "

LICENSE = "MIT"

inherit core-image

IMAGE_ROOTFS_SIZE ?= "8192"
IMAGE_ROOTFS_EXTRA_SPACE_append = "${@bb.utils.contains("DISTRO_FEATURES", "systemd", " + 4096", "" ,d)}"
~
```

图 7.10 编辑 core-image-minimal.bb

重新编译镜像 bitbake core-image-minimal, 编译成功后启动，hello 程序就可以正常启动了。

具体实现可参考官方文档:

https://docs.yoctoproject.org/4.0.9/dev-manual/common-tasks.html#writing-a-new-recipe

## 7.3 BSP 定制

用户添加自己的开发板到 Yocto , 遵循下面的步骤，以 E2000 为例:

1) 添加开发板配置

复制 phytium.conf（在 sources/meta-phytium/meta-bsp/conf/machine 文件夹下）,重命名为 <custom-board>.conf (用户自定义开发板名字）

2) 如需添加用户自己的内核补丁 kernel.patch（请根据实际名修改）, 复制 kernel.patch 到 sources/meta-phytium/meta-bsp/recipes-kernel/linux/linux-phytium 文件夹下，在 sources/meta-phytium/meta-bsp/recipes-kernel/linux/linux-phytium.inc 文件中添加:

```
SRC_URI:append:<custom-board>= " file://kernel.patch"
```

3) 添加后设置环境变量

```
$ source setup-env -m custom-board（用户自定义的开发板名字），后续的编译操作等请参考第 4 章。
```

Yocto 官方文档:

https://docs.yoctoproject.org/4.0.9/ref-manual/index.html

### 7.4 软件包管理

默认包管理是 rpm ,也可以使用 debain 和 opkg 软件包管理：

1) Dnf：RPM 软件包管理工具

添加下列变量到 conf/local.conf

```
PACKAGE_CLASSES = "package_rpm"
EXTRA_IMAGE_FEATURES += "package-management"
```

2) OPKG 软件包管理：

添加下列变量到 conf/local.conf

```
PACKAGE_CLASSES = "package_ipk"
EXTRA_IMAGE_FEATURES += "package-management"
```

3) APT：deb 软件包管理

添加下列变量到 conf/local.conf

```
PACKAGE_CLASSES = "package_deb"
EXTRA_IMAGE_FEATURES += "package-management"
```

## 8 常用软件

### 8.1 图形和显示

1) X11：core-image-xfce 文件系统，支持 X11 协议。
2) Wayland/Weston：core-image-weston 文件系统支持 Wayland 协议。
3) QT 图形： core-image-xfce 文件系统里,测试用例在/usr/share/examples 。core-image-weston 文件系统里,测试用例在/usr/share/examples/wayland。

### 8.2 多媒体

core-image-xfce 文件系统，默认不支持 VPU。

core-image-xfce 文件系统默认安装了 Gstreamer 来播放音频视频。还需要安装 VPU 包后，才能使用下面的命令。

1) 文件系统里显示 GST-OMX 提供的 GStreamer 信息：

```
$ gst-inspect-1.0 | grep omx
omx: omxrealvideodec: OpenMAX-IMG REAL Video Decoder
```

```
omx:  omxsorensondec: OpenMAX-IMG Sorenson Spark Video Decoder
omx:  omxvp8dec: OpenMAX VP8 Video Decoder
omx:  omxvp6dec: OpenMAX VP6 Video Decoder
omx:  omxvc1dec: OpenMAX WMV Video Decoder
omx:  omxmjpegdec: OpenMAX MJPEG Video Decoder
omx:  omxh263dec: OpenMAX H.263 Video Decoder
omx:  omxmpeg2videodec: OpenMAX MPEG2 Video Decoder
omx:  omxmpeg4videodec: OpenMAX MPEG4 Video Decoder
omx:  omxh265dec: OpenMAX-IMG HEVC Video Decoder
omx:  omxh264dec: OpenMAX H.264 Video Decoder
```

2) 用 gstreamer playbin 播放本地音频视频文件：

```
$ gst-launch-1.0 -v playbin uri=file:///<video>.mp4 video-sink=glimagesink
```

3) 用 gst-launch 构造管道，硬件解码播放本地视频文件：

```
$ gst-launch-1.0 -v filesrc location=<video>.mp4 ! qtdemux ! h264parse ! omxh264dec !
glimagesink
```

4) 硬件解码播放摄像头推送的网络流，比如以 rtsp 协议传输、h264 格式编码的视频流：

```
$ gst-lauch-1.0 rtspsrc location="<rtsp-address>" ! rtph264depay ! h264parse !
omxh264dec ! glimagesink
```

# 9 附录

## 9.1 开发主机交叉编译环境配置

交叉编译是在一个平台生成另一个平台的可执行代码，目前常见的是在 x86 平台生成其他平台的代码，如 x86 下生成 arm64 的二进制文件，目前飞腾支持基于 Yocto 和 Arm 的交叉编译工具链。

### 9.1.1 基于 Yocto 生成交叉编译工具链

飞腾提供的 SDK 包可构建基于 Yocto 的交叉编译工具链，构建安装步骤如下所示。

a) 进入飞腾 SDK 包目录，配置环境变量：

```
$ source setup-env -m <machine>
```

b) 编译交叉工具链：

```
$ bitbake meta-toolchain
```

c) 编译完成后进入构建目录下的 tmp/deploy/sdk

d) 运行 phytium-glibc-x86_64-meta-toolchain-aarch64-toolchain-<version>.sh，根据提示安装交叉编译器，一般选默认配置即可。

e) 假设交叉编译器安装目录为/opt，使用前按照如下命令配置即可使用。

```
$ source /opt/phytium/<version>/environment-setup-aarch64-phy-linux
```

f) 编译 linux 内核：

```
$ cd <linux-source-path>
$ make phytium_defconfig
$ make
```

### 9.1.2 下载交叉编译工具链

Arm GNU 编译工具链，为基于 Linux 内核的开发者提供了稳定得编译环境。交叉编译工具链官网下载地址如下:

https://developer.arm.com/-/media/Files/downloads/gnu/11.3.rel1/binrel/arm-gnu-toolchain-11.3.rel1-x86_64-aarch64-none-linux-gnu.tar.xz

配置步骤：

a) 解压到/opt 目录

```
$ tar xJf arm-gnu-toolchain-11.3.rel1-x86_64-aarch64-none-linux-gnu.tar.xz -C /opt
```

b) 配置环境变量和编译内核

```
$ sudo apt install flex bison libssl-dev
$ export PATH=/opt/arm-gnu-toolchain-11.3.rel1-x86_64-aarch64-none-linux-gnu/bin:$PATH
$ export ARCH=arm64 CROSS_COMPILE=aarch64-none-linux-gnu-
$ export CC=aarch64-none-linux-gnu-gcc
$ cd <linux-source path>
$ make phytium_defconfig
$ make
```

## 9.2 常见问题

1) 支持调试

为了支持调试，可以添加下列规则到 local.conf 文件中：

```
IMAGE_FEATURES:append = " tools-debug dbg-pkgs"
```

2) 更改内核组件选项

运行下列命令：

```
$ bitbake -c menuconfig virtual/kernel
$ bitbake -c compile virtual/kernel (只运行编译任务 )
```

3) 内核源码位置：

```
build_phytium/tmp/work/phytium-phy-linux/linux-phytium/4.19-r0/git
```

4) 重新编译软件包

用户重新编译软件包，运行下列命令：

```
$ bitbake -c cleansstate < package name>
```

清除编译的输出结果和 shared state cache

```
$ bitbake < package name>
```

编译 package

5) 编译生成的镜像位置：

```
tmp/deploy/images/phytium/
```

6) qt x11 去掉桌面

添加下列变量到 local.conf

```
IMAGE_FEATURES:remove = "x11-sato"
RDEPENDS_${PN}:remove = "matchbox-terminal"
```

7) 内核打补丁

添加用户自己的内核补丁 kernel.patch（请根据实际名修改），复制 kernel.patch 到 sources/meta-phytium/meta-bsp/recipes-kernel/linux/linux-phytium 文件夹下。

需要修改 sources/meta-phytium/meta-bsp/recipes-kernel/linux/linux-phytium.inc 文件。

如果补丁是针对指定开发板用如下规则：

```
SRC_URI:append:<custom-board>= " file://kernel.patch"
```

如果补丁适用所有开发板用如下规则：

```
SRC_URI:append = " file://kernel.patch"
```

8) 编译应用开发标准 SDK

对于 core-image-xfce，运行：

```
$ bitbake -c populate_sdk core-image-xfce
```

对于 core-image-weston：

添加下列变量到 build_<board>/conf/local.conf：

```
DISTRO_FEATURES:remove = " x11"
```

然后运行：

```
$ bitbake -c populate_sdk core-image-weston
```

9) 编译应用开发可扩展 SDK

对于 core-image-xfce：

添加下列变量到 build_<board>/conf/local.conf：

```
SDK_INCLUDE_BUILDTOOLS =""
```

然后运行：

```
$ bitbake -c populate_sdk_ext core-image-xfce
```

对于 core-image-weston：

添加下列变量到 build_<board>/conf/local.conf：

```
DISTRO_FEATURES:remove = " x11"
SDK_INCLUDE_BUILDTOOLS ="""
```

然后运行：

```
$ bitbake -c populate_sdk_ext core-image-weston
```

10) 编译 qt5 交叉工具链

```
$ bitbake meta-toolchain-qt5
```

11) 使用指定内核源代码

添加下列变量到 build_<board>/conf/local.conf

```
INHERIT += "externalsrc"
EXTERNALSRC:pn-linux-phytium = "path-to-your-source-tree"
```

内核代码需要 git commit 信息,内核源代码如果没有,需要如下操作

```
$ cd <path-to-your-source-tree>
$ git init
$ git add .
$ git commit -s -m “init”
```

## 9.3 参考

https://docs.yoctoproject.org/4.0.9/

# A 更新记录

| 发布日期 | 版本 | 说明 |
|---|---|---|
| 2022-08-11 | V1.0 | 初稿，针对飞腾 E2000 系列 |
| 2023-02-24 | V1.1 | 支持构建 xenomai 系统；支持制作安装盘，使用安装盘安装系统；VPU 的使用；支持 5.10 和 4.19 内核版本的更换 |
| 2023-03-24 | V1.2 | 删除 ISO 发布的二进制安装包，修改安装盘安装系统后的启动参数，增加开发环境的软件依赖包 |
| 2023-05-15 | V2.0 | Yocto 升级到 4.0，文件名改为“飞腾嵌入式 Linux Yocto4.0 用户手册” |
| 2023-08-09 | V2.1 | 支持构建 jailhouse，ethercat；修改基于 VPU 的描述 |
| 2024-01-16 | V2.2 | 支持 phydisk.img ,添加编译和使用说明；统一 CPU 名字 |
| 2024-04-30 | V2.3 | 增加 E2000 VPU, X100 安装包说明 |
| 2024-10-25 | V2.4 | 增加 ethercat,支持 xenomai 选项 |
| 2025-05-30 | V2.5 | 增加外部工具链, Gnome 桌面支持 , 环境配置脚本的使用说明 |
| 2025-12-03 | V2.6 | 增加 D3000M 支持及 VPU,GPU 安装包说明；添加 Yocto 分支说明 |
| 2026-03-02 | V2.7 | 删除“此外，D3000M 仅在该分支下得到支持” |